Paradigmas de Programação — Aula 09: Modelo de atores com Elixir

`spawn`, `send` e `receive`

As três primitivas

Primitiva	Efeito
`spawn(fn -> ... end)`	cria um processo e devolve o seu **PID**
`send(pid, mensagem)`	deposita a mensagem na caixa de `pid` e retorna de imediato
`receive do ... end`	retira a **primeira mensagem que casa** com algum padrão; bloqueia se nenhuma casa
`self()`	PID do processo corrente

- **Recebimento seletivo**: mensagem que não casa fica na caixa, na mesma posição
- Processo de vida longa termina o `receive` com uma cláusula que casa com qualquer termo e descarta o que não reconhece

A resposta é outra mensagem

```elixir
defmodule Dobrador do
  def atender do
    receive do
      {:dobro, do_processo, numero} -> send(do_processo, {:resultado, numero * 2})
    end
  end
end

dobrador = spawn(fn -> Dobrador.atender() end)
send(dobrador, {:dobro, self(), 21})       # o remetente envia o próprio PID

receive do
  {:resultado, valor} -> IO.puts("resposta: #{valor}")   # resposta: 42
end
```

- O `Dobrador` passa a conhecer o `main` porque o PID chegou **como dado**
- `atender/0` não chama a si mesma: trata uma mensagem e termina

Prazo e ordem de chegada

```
receive do
  :nunca_chega -> IO.puts("nao acontece")
after
  200 -> IO.puts("nada chegou em 200 ms")
end
```

- `after`: o equivalente ao `select` com `time.After` da aula 08, sem canal de temporizador
- Três processos esperam 300, 100 e 200 ms e avisam o `main`: chegam **B, C, A** — a ordem do envio, não a da criação

Estado por recursão de cauda

```
def loop(valor) do
  receive do
    {:salvar, novo_valor, do_processo} ->
      send(do_processo, {:salvo, novo_valor})
      loop(novo_valor)              # chamada de cauda com o novo estado

    {:ler, do_processo} ->
      send(do_processo, {:valor, valor})
      loop(valor)
  end
end
```

- O estado é o **argumento**; nenhuma variável é alterada
- Chamada de cauda: a BEAM a executa como salto, a pilha não cresce
- É a terceira ação do ator (nota 01): definir o estado com que tratará a próxima mensagem

Calculadora e memória (`03-calculadora.exs`)

```
sequenceDiagram
    participant M as main
    participant C as calculadora
    participant Mem as memoria (valor)
    M->>C: {:soma, memoria, main, 7, 5}
    M->>C: {:subtracao, memoria, main, 20, 3}
    M->>C: {:multiplicacao, memoria, main, 3, 4}
    C->>Mem: {:salvar, 12, main}
    Mem-->>M: {:salvo, 12}
    C->>Mem: {:salvar, 17, main}
    Mem-->>M: {:salvo, 17}
    C-->>M: {:nao_entendi, :multiplicacao}
    M->>Mem: {:ler, main}
    Mem-->>M: {:valor, 17}
```

- Calculadora sem estado; memória guarda o último valor como argumento
- Tradução para Elixir de um exemplo do professor escrito originalmente em Erlang

Ordem entre remetentes distintos

- Garantia: mesmo remetente → mesmo destinatário, ordem do envio. Entre remetentes distintos, **nenhuma**
- `:salvar` vai da calculadora à memória; `:ler`, do `main` à memória: sem espera, o `:ler` poderia chegar antes de um `:salvar` e devolver 0 ou 12
- O `main` espera **uma resposta por operação** (`{:salvo, valor_salvo}` ou `{:nao_entendi, operacao}`) antes de ler
- O original esperava 100 ms com `sleep`: espera por tempo torna a ordem provável, não garantida

```
memoria antes: 0
calculadora: 7 + 5 = 12
calculadora: 20 - 3 = 17
main: memoria confirmou 12
main: memoria confirmou 17
calculadora: nao entendi multiplicacao
main: calculadora recusou multiplicacao
memoria depois: 17
```

A intercalação das linhas varia entre execuções; o valor final, 17, não.

Processos que caem: `Process.monitor`

```
referencia = Process.monitor(fragil)
send(fragil, :falhar)             # fragil levanta "falha simulada"

receive do
  {:DOWN, ^referencia, :process, _processo, {motivo, _pilha}} ->
    IO.puts("o processo caiu: #{Exception.message(motivo)}")
end
IO.puts("o main continua vivo")
```

- Exceção não tratada termina **só** o processo que a levantou
- O monitor entrega `{:DOWN, referencia, :process, pid, motivo}` ao observador; `^referencia` fixa o valor no padrão
- A queda chega a outro processo como mensagem: reagir a ela criando um substituto é a ideia do **Supervisor** (nota 04)